

# Distributed Systems - 1\_10

Today's we'll see Berkley algorithm, it's an internal clock synchronization algorithm.

By taking any pair of clock, we'll have that the Skew is bounded by  $D$ .

We use a master-slave structure, with which we have a single entity known as the master/leader. He ensures with some actions to make the system synchronized.

The idea, in this case, is to have the master (like Computer 1), and he periodically send a broadcast message to the entire system, asking the processes for their clock. Everyone will answer with their clock, then we'll compute an average value. The Master  $M$  will compute Deltas for each computer.

Let's see an example.

We have the master, with a time of 3:05, then we have slave 1 with 2:55, then slave 2 with 3:00 and slave 3 with 3:25. The master asks the time for everyone, receiving the answers. The master will now compute the delta between the master and itself, which is obviously zero, then He will do the same for  $\Delta_{M1} = -10$ , then  $\Delta_{M2} = -5$  and then  $\Delta_{M3} = 20$ . By using those quantities, we'll compute an average difference, but before doing so the masters will remove processes who are likely faulty, so processes who surpass a certain threshold. By setting the threshold, for example, at 10, we will remove  $\Delta_{M3}$ , because it's  $20 > 10$ . We will now have that  $Avg = \frac{0-10-5}{3} = -5$ .

We'll tell anyone now that, in order to move to the correct value, the process will have to move to  $\Delta_{Mi} - Avg$ . For example, for the slave 1 he'll have to move to  $-5 - (-10) = +5$ . The slave 2 will have to move of  $-5 - (-5) = 0$ . We compute also for slave 3, by having  $-5 - 20 = -25$ . Also for the master, by having  $-5 - 0 = -5$ . Each process will now be at 3:00.

*For simplicity here we can go back in time, though in real life we should slow down the clock.*

The accuracy of Berkley depends on the maximum round trip time.

It's not a fault tolerant algorithm for the master. If the slaves are faulty then we'll fix everything. Though if the master is faulty (like it crashes), the synchronization stops working. What we do is elect a new master, known as "leader election".

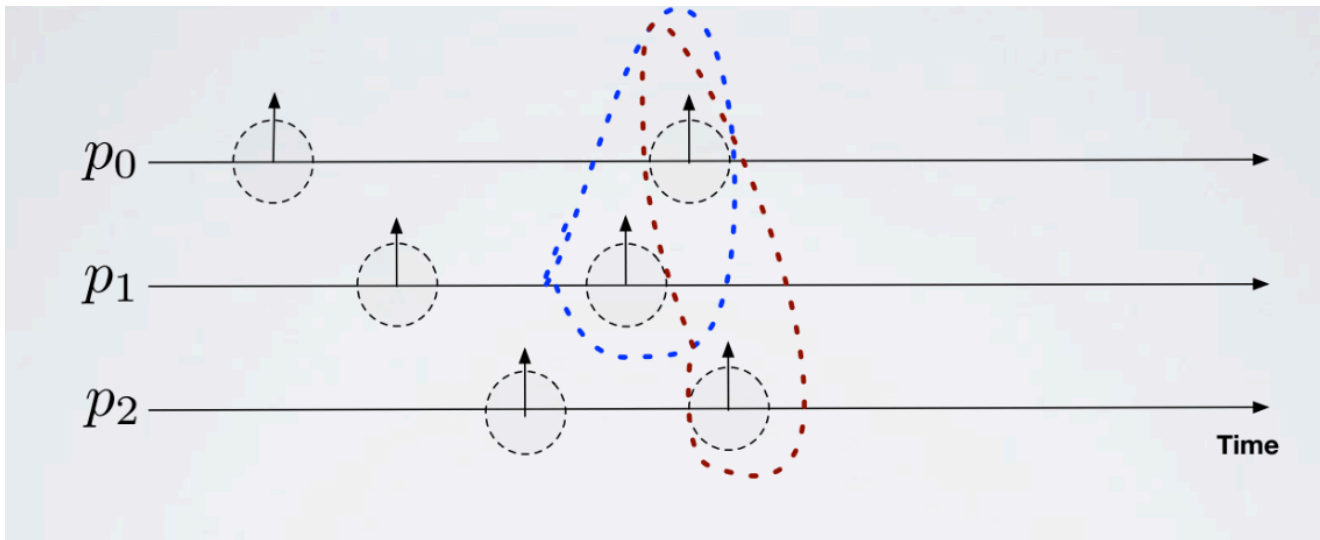
*It's not tolerant with byzantine failures.*

In real life we use NTP (Network Time Protocol), it synchronizes computers with UTC.

The servers are organized in a hierarchy. The server at the top has the clock. The second level (stratum) synchronize directly with server 1, with an algorithm similar to Cristian's one. The stratum 3 only synchronize with stratum 2. *We can have more stratum, like 4 and so on.* Error propagates with the descension of stratum (so stratum 3, for example, will have a cumulative error of  $\mathcal{E}_1$  and  $\mathcal{E}_2$ ). *We do so so that the bandwidth of the server 1 is not completely saturated.*

Clock synchronization has a limit, we can do it only on synchronous system, we have to have predictable delays in the channel.

Even on synchronized systems, we could have a problem with clocks, given by the bounded accuracy. The problems take place when the bound of the error overlaps with the bound of the error of another process, by not guarantee the order.



*When at the exam we are told that processes are synchronized, we know that this problem will not happen. Though how do we take care of it in real life? We'll take a look at total order algorithms.*

## Logical time

Logical time is a concept of time we can implement also in asynchronous systems. It takes care of the causal relationship between events. We'll talk about two methods given by Scalar/Lamport's clocks, and then about Vector Clocks, used to measure logical time. We'll also see Lamport's Mutual Exclusion algorithm.

Let's do an example.

Let's say we want to build a chat application. The easiest thing to do would be using a server-client structure. If we are in a scenario with crash failures, if the server dies the application stops working. Another way is not using a server, *i got lost sorry.*

Let's say we have a chat application with a specific pattern of messages. We have Alice asking a question, then Mike responds, then Alice asks another question and then another entity answers. This would be the real order of messages.

Because of the fact that the system is asynchronous, a broadcast message might arrive with a certain delay to other entities.

Let's say that Bob, at a certain time  $t$ , only receives the messages from Mike and Bob's supervisor, and not those from Alice. Bob would have a limited local view. To avoid this kind of problem we have to formalize the concept of causality (the messages of Mike, ecc are caused by Alice sending the first message).

The simple FIFO ordering, in this situation, will not help us. Our goal is to find a way to timestamp an event, and to do that we have to formalize the concept of causality, and then we timestamp events.

The definition of casual relationship was introduced by Lamport. We say that an event happens before one another if there is a casual relationship between these two.

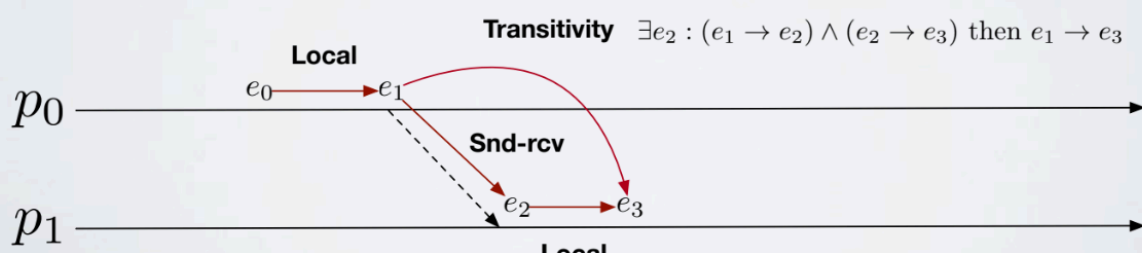
Let's say that I have  $p_0$ , and two events happen on it,  $e_0$  and then  $e_1$ . In this case there is a casual relationship that  $e_0$  happens before  $e_1$  (we write it with an arrow).

With condition B, let's say we have  $e_1$  that's the sending of message  $m$ ,  $e_2$  the receival. We say that  $e_1$  happens obviously  $e_2$  (we received a message because someone has sent it).

We can now define a partial ordering between events in the system, we introduce three properties:

Two events  $e$  and  $e'$  are related by happened-before relation ( $e \rightarrow e'$ ) if:

- Local ordering:  $\exists p_i \mid e \rightarrow_i e'$
- Snd-rcv ordering:  $\forall m, \text{send}(m) \rightarrow \text{receive}(m)$ 
  - $e = \text{send}(m)$  is the event of sending a message  $m$
  - $e' = \text{receive}(m)$  is the event of receipt of the same message  $m$
- Transitivity:  $\exists e'' : (e \rightarrow e'') \wedge (e'' \rightarrow e') \text{ then } e \rightarrow e'$ 
  - the *happened-before* relation is transitive

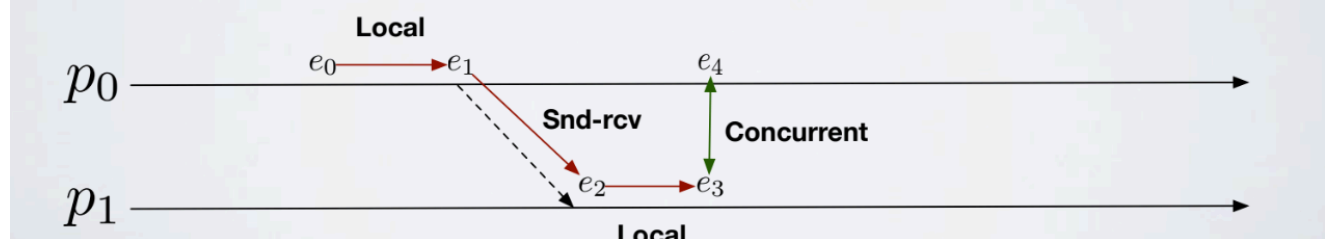


The first property is the local ordering, (A) in slides, and the second receive is (B). For the first, for example, we say that  $e$  and  $e'$  are related if a process  $p_i$  exists by which  $e \rightarrow_i e'$  ( $e$  is related with  $e'$  in  $p_i$ ).

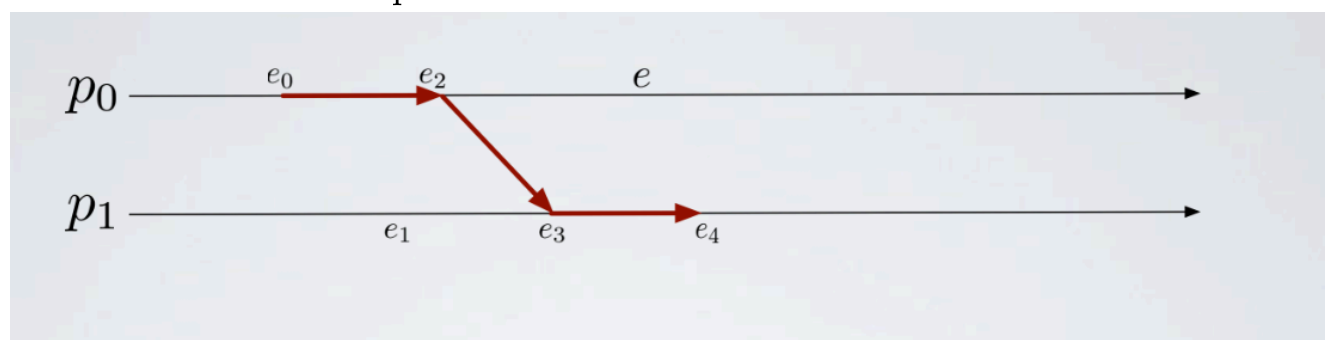
Then we have transitivity. This relationship (in the photo) is telling us that  $e_3$  knows about  $e_0$ .

This relationship is partial, we could have events related by "happened before", Let's say we have on  $p_0$  another process  $e_4$  happening. We say  $e_3$  and  $e_4$  are concurrent. Two events are concurrent if they do not know about each other. The only way to have them know is having a message from  $p_0$  to  $p_1$  or viceversa.

- The sequence  $e_1, e_2, \dots, e_n$  may not be unique
- It may exists a couple of events  $\langle e_1, e_2 \rangle$  such that  $e_1$  and  $e_2$  are not in happened-before relation
- If  $e_4$  and  $e_3$  are not in happened-before relation then they are **concurrent** ( $e_4 || e_3$ )
- For any two events  $e_x$  and  $e_y$  in the execution history of a distributed system, either  $e_x \rightarrow e_y$ ,  $e_y \rightarrow e_x$  or  $e_y || e_x$



Let's take another example:

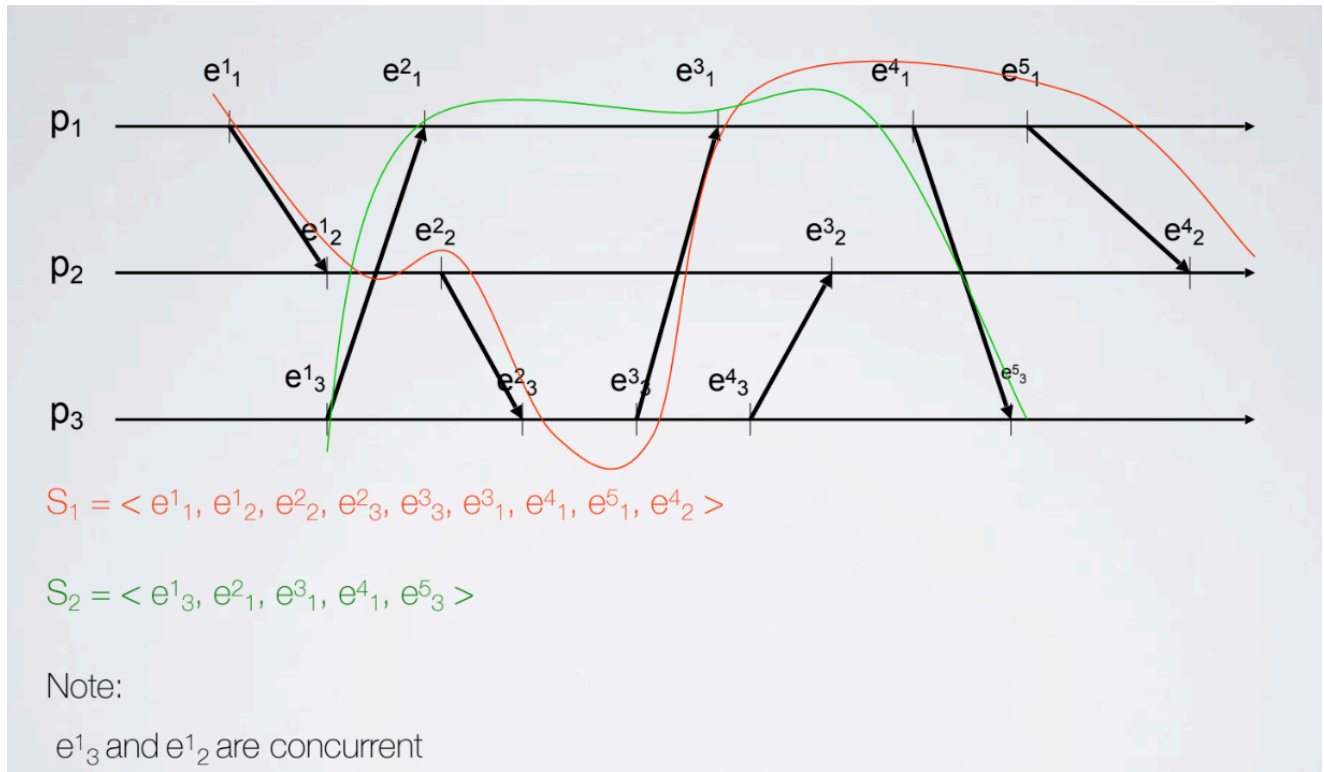


Here we have that  $e_1$  and  $e$  are concurrent. Even between  $e_3$  and  $e$ .

The "happened before" relationship is a poset.

This relates to the concept of "indistinguishably" we've seen before. Intuitively we can move concurrent events and have an execution that is not distinguishable.

Let's take another example.



*To see if two events are concurrent or not, we could try to find a path between them.*

Let us start with the first tool, the Logical Clock (Lamport's Clock).

We attach timestamp to each event, and this number will have a special property, indicated as  $L_i(e)$ . We have that if  $e$  happens before  $e'$ , then  $L(e) < L(e')$ . (Only works with  $\implies$ , not with  $\leftarrow$ ). If the logical clock of  $e$  is smaller than  $e'$ , then this doesn't imply that  $e$  has happened before  $e'$ , they might be concurrent.

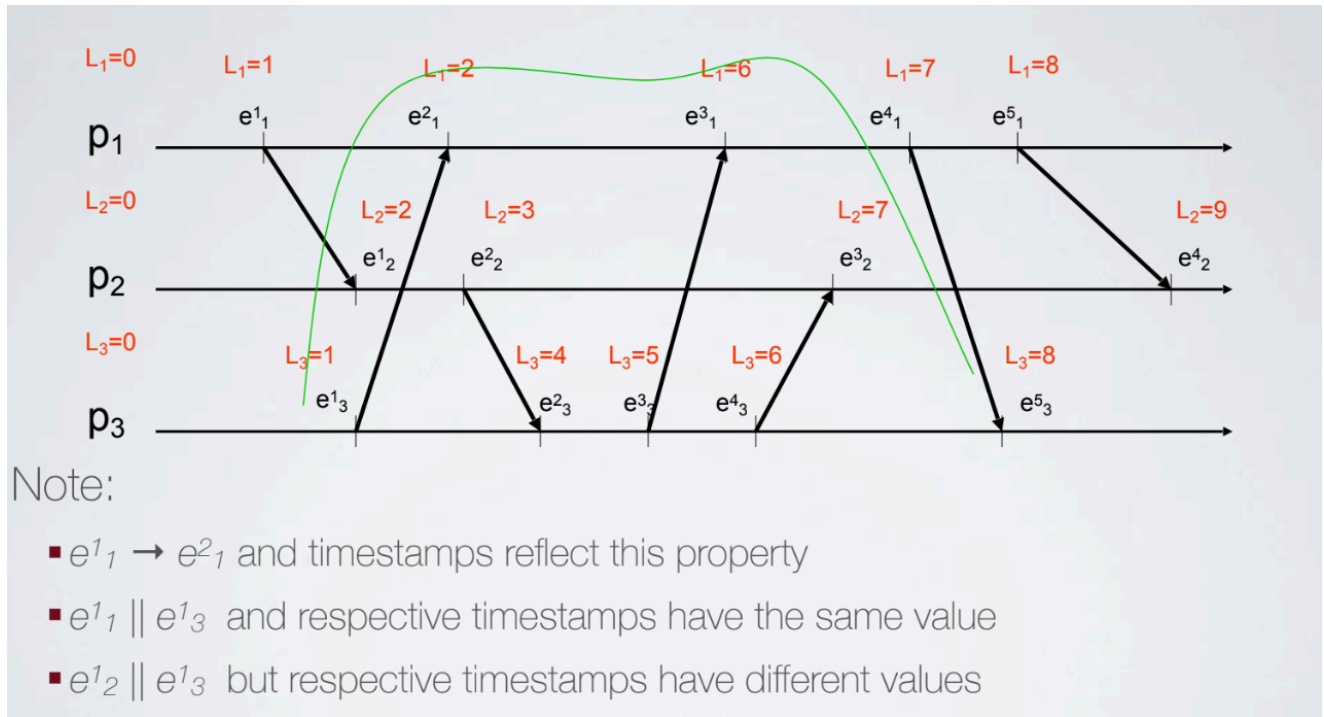
With the logical clock everyone starts with a counter  $L_i$ , initially set to 0. We then increase the counter, by following two rules:

1. when a local event happens, we increase the counter by 1
2. when we send a message, we also increase the counter by 1. Along side sending the message, we also send the value of the counter ( $(m, 2)$  for example).

When I receive a message as process, we'll do a  $1 + \text{Max}(L, L(m))$  (maximum between our logical clock and the value sent with the message), setting our counter to that value.

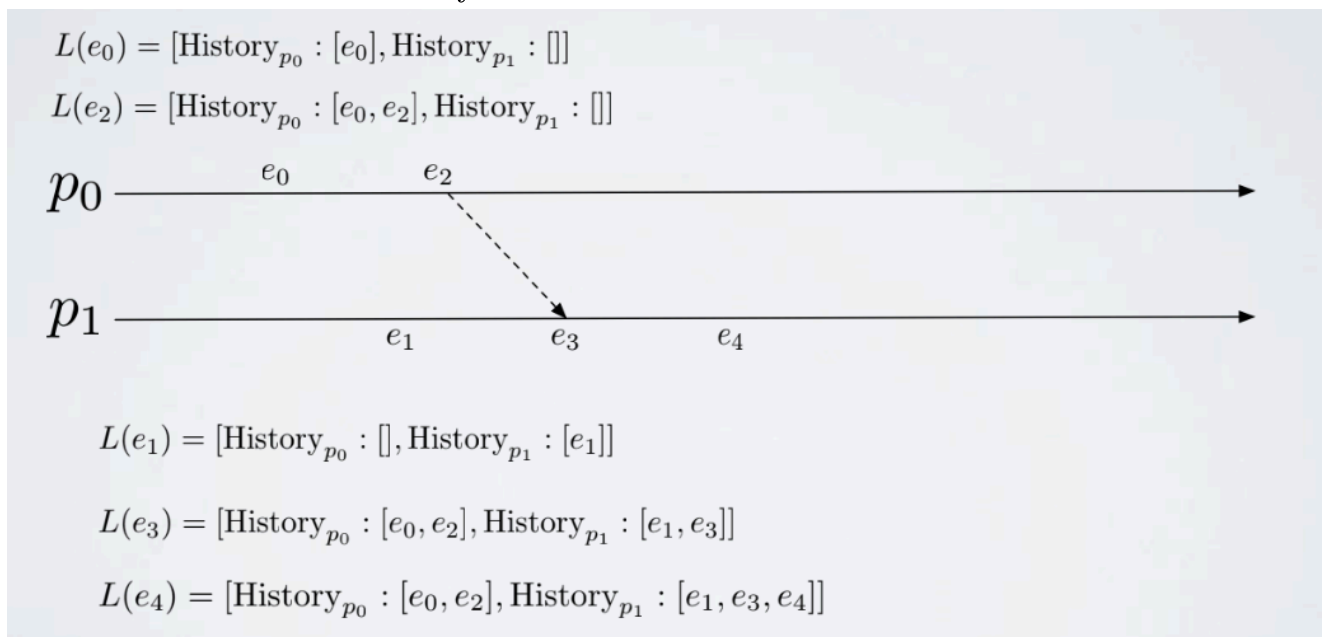
We have to show that  $e \rightarrow e'$  ( $e$  happens before  $e'$ ), then  $L(e) < L(e')$ . If  $e$  happens before  $e'$ , we have a path between  $e$  and  $e'$ . Whether the possibility, we increase by at least one the value of the logical clock. Then if  $e$  happens before  $e'$ , we'll have that  $L(e')$  is greater at least by 1 than  $L(e)$ .

Let's take an example:



We would like to have now the  $\iff$  for the relationship of local clock. There exists Vector Clock, which allows us to design a system which has the other direction of the property. We do not have a simple number, but a vector. By comparing vectors, we'll be able to tell if an event happen before another.

As step 1 we construct this vector as a list of events, as step 2 we transform this list of events in a vector. We attach then a data structure attached to an event, which summarizes the history of the events.



We'll have one entry (in the data structure) for each process.

At first the data structure will be  $[[], []]$  both for  $p_0$  and  $p_1$ . When process zero creates an event  $e_0$ , it's history will change. At this point we still do not know nothing

about  $e_1$ . Same goes for  $e_1$  for  $p_1$ .

The local event is trivial, the "problem" is when we send a message. The message will include the history of  $p_0$  and  $p_1$  (which at  $e_2$  it's still empty). When  $p_1$  receives the message, he will now know the history of  $p_0$  and its as well. We'll fill the history of  $p_0$  with the longest one (received with the message), and the history of  $p_1$  will be made by  $e_1$  and  $e_3$ .

What we are doing is that we are constructing the view of the process. Now, how do we compare data structures? We say that  $L(e) < L(e')$  if, for each location of the history, the list is smaller than the other. As example of history of  $e_0$  and  $e_3$ . Some data structures are not comparable, for example  $e_0$  and  $e_1$ , as we have  $[[e_0], []]$  for  $e_0$  and  $[[], [e_1]]$ . For the first location  $p_0$  is winning, but then for the second one  $p_1$  is winning, they are not comparable. This happens because the events are concurrent.

Let's see the proof.

We have to show that if  $e \rightarrow e'$  then  $L(e) < L(e')$  (data structure of  $e$  is contained in  $e'$ 's one). If  $e$  happens before  $e'$ , then there is a path between  $e$  and  $e'$ , then the data structure at  $e'$  will contain the one of  $e$ .

How to show the other direction? If the data structure of  $e$  is smaller than  $e'$ , then  $e \rightarrow e'$ . If the data structure of  $e$  is less than  $e'$ , then  $L(e)$  is contained in  $L(e')$ , then all events happened at  $e$  then they are contained in  $L(e')$ . Because of the fact that  $e$  is contained also in  $L(e')$ , then there must be a path that somehow has connected  $e$  to  $e'$ , then  $e \rightarrow e'$ .

This data structure captures causality.

What are the disadvantages of using this data structure? The use of a large amount of memory for a lot of events. How do we get something more compact?

Let's say that on my system I find two events,  $e$  and  $e'$ . I examine the data structure of  $e$  and  $e'$  and I find that, for a certain location  $x$  in the history, I find something similar.

To have something different would be impossible (like in the slide's example)

$$\exists e : L(e) = [\dots, History_{p_x} = [e_0, e_1, e_2], \dots]$$

$$\exists e' : L(e') = [\dots, History_{p_x} = [e_0, e_1, e_3], \dots]$$

**Possible?**

The only one who can append something at the history of  $p_x$  is  $p_x$  itself, so the last event can't be different.

What matters is not the content of the list, but the size of the list. We can then compress the data structure by just using the size of the list.

So at first we start with a vector for all zeros, having empty lists for each process. Let's say we are  $p_1$  and we create a new event, then I increase the value of  $+1$  for the size of  $p_1$ . When another process receives a message, it compares the elements between the lists and take the maximum.

We can say that one vector is less than the other ( $V(e) < V(e')$ ) if all components are  $\leq$  and there is one component who is strictly less.